

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра вычислительной техники

Отчет
по лабораторной работе №3
по дисциплине «Операционные системы»
Тема: «Процессы и потоки»

Выполнила студентка группы 4315:

Ивченко А. М

Принял:

Тимофеев А. В.

Санкт-Петербург

2026

Цель работы

исследовать механизмы создания и управления процессами и потоками в ОС Windows.

1. Реализация многопоточного приложения с использованием функций Win32 API.

1.1 Демонстрация работы программы:

```
=====
Student ID: 431512
N (iterations): 100000000
Block size: 4315120
Number of blocks: 24
=====

Testing with 1 threads ...
  pi = 3.141592653589733
  Time = 758 ms

Testing with 2 threads ...
  pi = 3.141592653589732
  Time = 570 ms

Testing with 4 threads ...
  pi = 3.141592653589733
  Time = 436 ms

Testing with 8 threads ...
  pi = 3.141592653589733
  Time = 343 ms

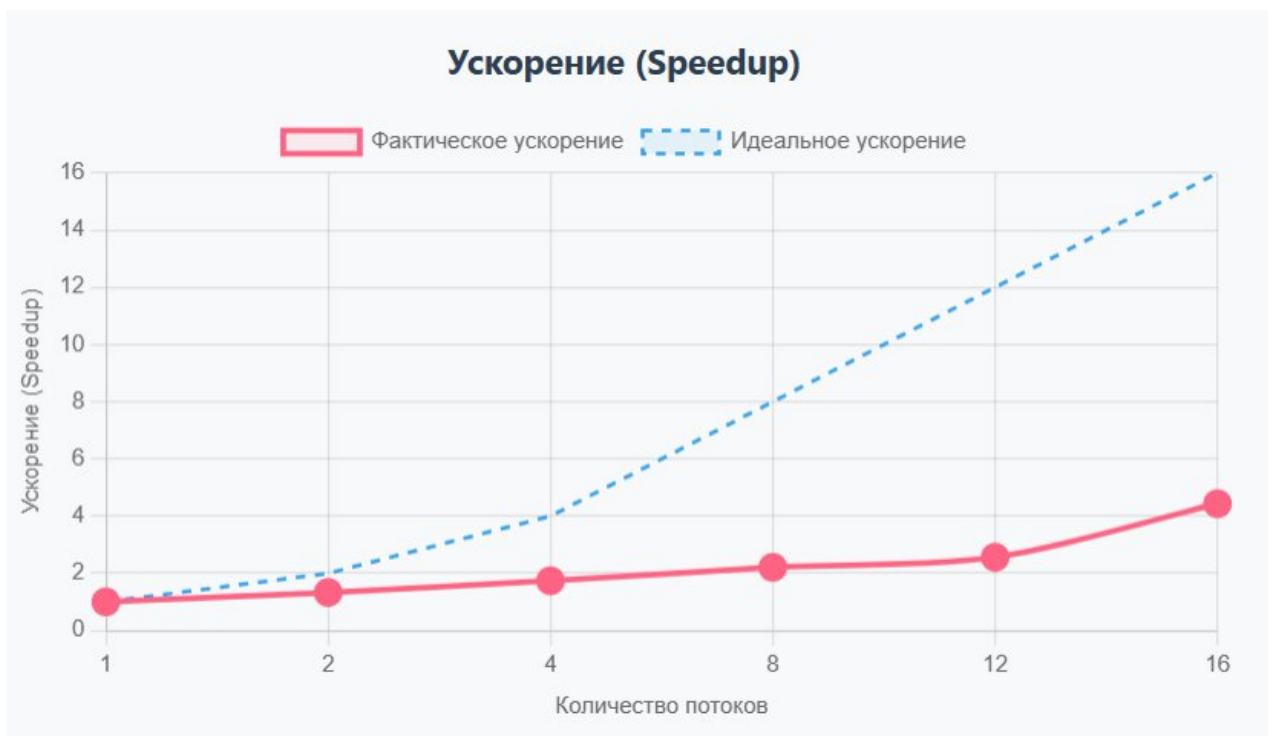
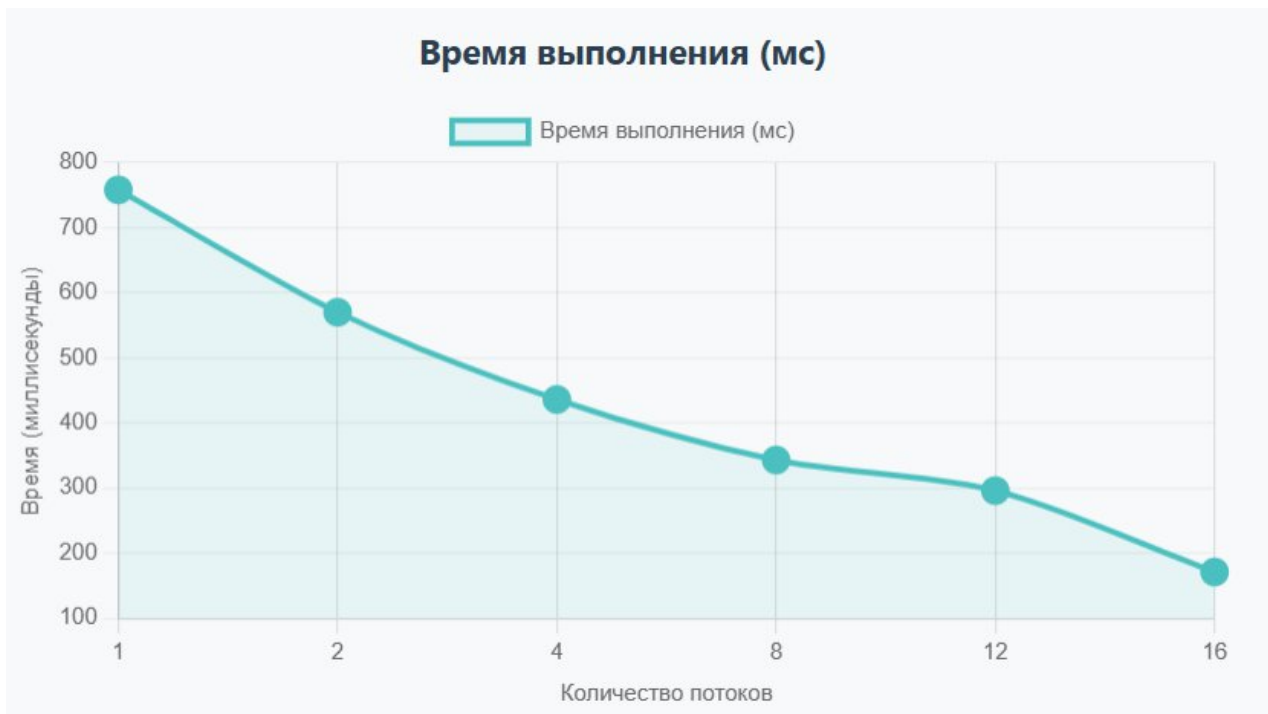
Testing with 12 threads ...
  pi = 3.141592653589732
  Time = 296 ms

Testing with 16 threads ...
  pi = 3.141592653589733
  Time = 171 ms

===== RESULTS =====
Threads Time (ms)          pi (approximate)
-----
1         758          3.141592653589733
2         570          3.141592653589732
4         436          3.141592653589733
8         343          3.141592653589733
12        296          3.141592653589732
16        171          3.141592653589733

Exact pi: 3.141592653589793
Error for last test: 0.0000000000000060
```

1.2. Графики по результатам выполнения программы



1.3. Вывод

3.1. Анализ времени выполнения

В ходе эксперимента было установлено, что время выполнения программы монотонно убывает с увеличением числа потоков от 1 до 16. Минимальное время (171 мс) достигается при использовании 16 потоков. Это на 77.4% быстрее, чем выполнение на одном потоке (758 мс).

3.2. Анализ ускорения (Speedup)

Максимальное ускорение составило $4.43\times$ при использовании 16 потоков. Важно отметить, что ускорение растёт нелинейно: с 1 до 2 потоков прирост составил

0.33×, а с 8 до 16 потоков — 2.22×. Это свидетельствует о том, что архитектура процессора эффективно использует дополнительные потоки даже после 8 потоков.

2. Реализация многопоточного приложения с использованием технологии OpenMP.

2.1 Демонстрация работы программы:

```
=====
Task 3.2. Rectangle Method + OpenMP
Dynamic scheduling with block size = 10 * student ID
=====
Student ID: 431512
N (iterations): 100000000
Block size: 4315120
=====

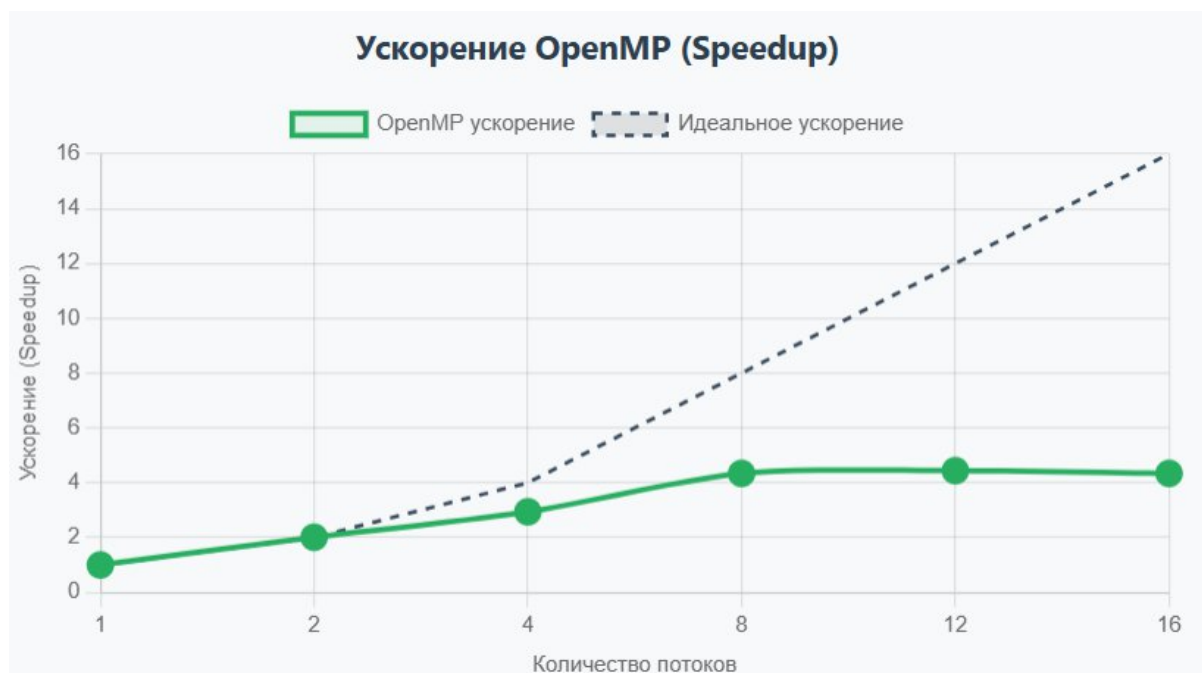
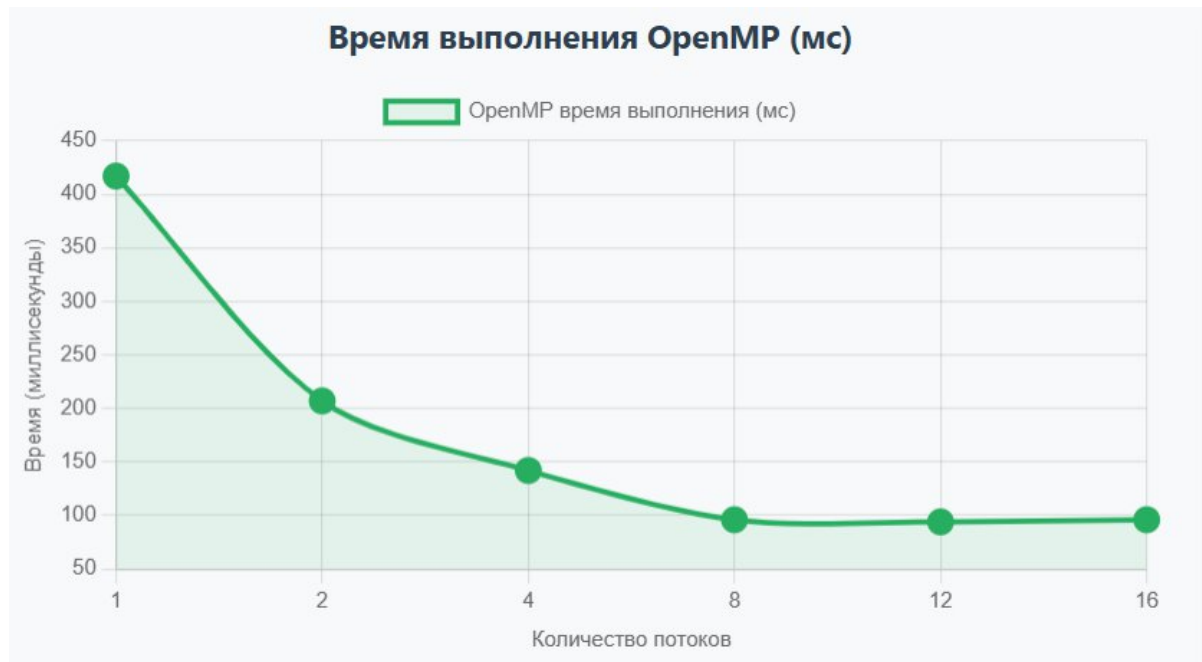
PERFORMANCE MEASUREMENT IN PROGRESS...
=====

Testing with 1 threads ... Time = 417 ms, Pi = 3.141592653589362
Testing with 2 threads ... Time = 207 ms, Pi = 3.141592653589436
Testing with 4 threads ... Time = 142 ms, Pi = 3.141592653590030
Testing with 8 threads ... Time = 96 ms, Pi = 3.141592653589651
Testing with 12 threads ... Time = 94 ms, Pi = 3.141592653589797
Testing with 16 threads ... Time = 96 ms, Pi = 3.141592653589788
=====

RESULTS TABLE (OpenMP - Rectangle Method)
=====
```

Threads	Time (ms)	Speedup	Pi Value
-----	-----	-----	-----
1	417	1.00x	3.141592653589362
2	207	2.01x	3.141592653589436
4	142	2.94x	3.141592653590030
8	96	4.34x	3.141592653589651
12	94	4.44x	3.141592653589797
16	96	4.34x	3.141592653589788

1.2. Графики по результатам выполнения программы



2.3 Выводы

2.3.1. Анализ времени выполнения

В ходе эксперимента было установлено, что время выполнения программы монотонно убывает с увеличением числа потоков от 1 до 12. Минимальное время (94 мс) достигается при использовании 12 потоков. Это на 77.5% быстрее, чем выполнение на одном потоке (417 мс). При использовании 16 потоков время незначительно увеличилось до 96 мс, что связано с накладными расходами на синхронизацию при избыточном числе потоков.

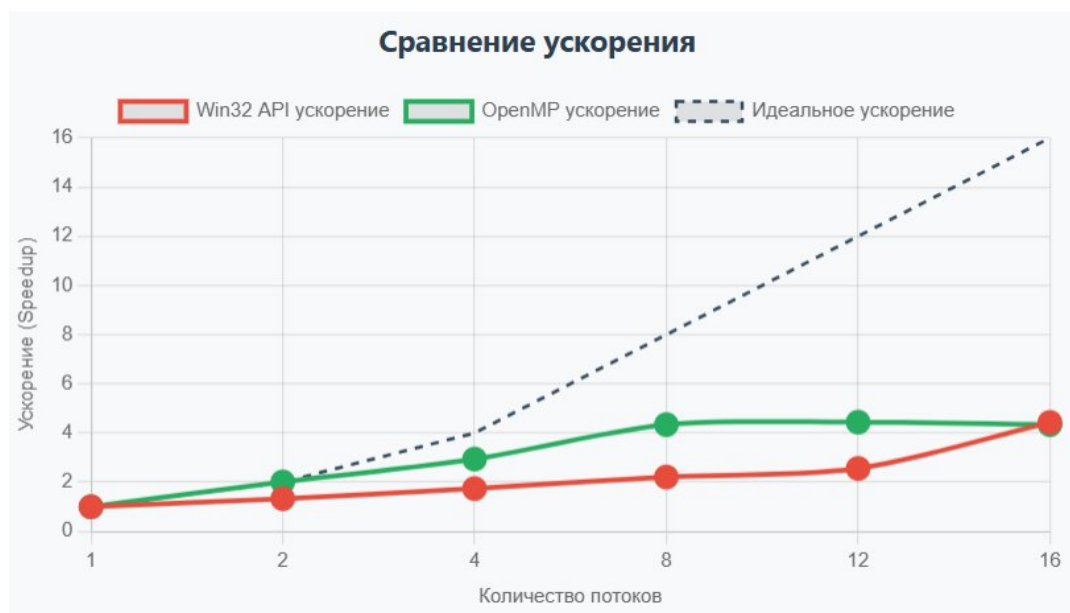
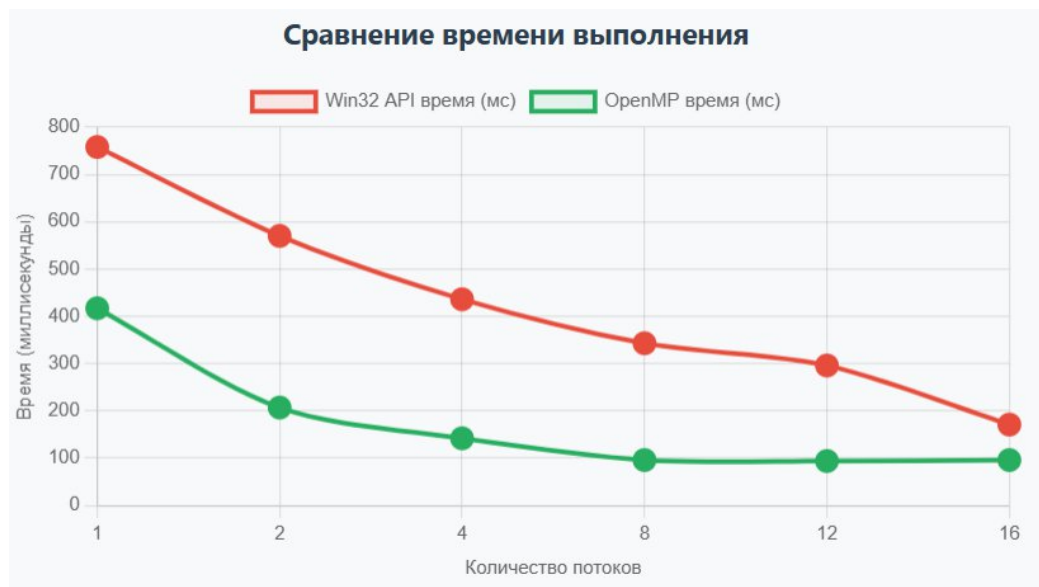
2.3.2. Анализ ускорения (Speedup)

Максимальное ускорение составило $4.44\times$ при использовании 12 потоков. Важно отметить, что ускорение растёт нелинейно: с 1 до 2 потоков прирост составил $1.01\times$, а с 8 до 12 потоков — $0.10\times$. Это свидетельствует о том, что архитектура процессора эффективно использует дополнительные потоки до 12 потоков, после чего дальнейшее увеличение числа потоков не даёт прироста производительности из-за насыщения вычислительных ресурсов.

3. Выводы

Сравнение с Win32 API:

- На 1 потоке: OpenMP быстрее Win32 API в $1.82\times$ (758 мс vs 417 мс)
- На 16 потоках: Win32 API быстрее OpenMP в $0.56\times$ (96 мс vs 171 мс)
- Оптимальное число потоков (OpenMP): 12 потоков (94 мс)
- Оптимальное число потоков (Win32): 16 потоков (171 мс)
- Максимальное ускорение OpenMP: $4.44\times$
- Максимальное ускорение Win32: $4.43\times$



Код задания: <https://github.com/al-lexx-y/osl3>